

Lorem ipsum

Grobe Struktur:

1 Einleitung

Transformer-Architekturen haben sich in den letzten Jahren vor allem im Natural-Language-Processing bewährt, dabei ist diese Modell-Klasse vielseitig einsetzbar [Hier Zitate von anderen Domänen]. Auch für unser Problem, der Generierung einer Sequence, kann eine Transformer-Architektur genutzt werden.

Transformer-Modell seit ChatGPT sehr bewährt, blibliablu

2 Theorie

[Gleichheiten zu RNNs]. Auch hier wollen wir $N = 128$ Noten als Input liefern und die nächsten $O = 1$ vorhersagen lassen. Die Folge $(x_1, x_2, \dots, x_{N+1})$ geben wir dann wieder in das Modell und erhalten so schließlich das generierte Stück. Wie auch im Fall der RNNs ist unser Preprocessing denkbar einfach, wieder 88 Noten. Nur fügen wir nun noch eine SOS- und EOS-Token ein (die Abkürzungen stehen für Start-of-Sequence bzw. End-of-Sequence). Unser "Vokabular" hat somit Größe 90; der SOS-Token dient dazu, dass Modell später überhaupt zum Generieren zu bringen. Der EOS-Token bietet den Vorteil, dass das Modell selbst entscheidet, wann das Stück fertig komponiert ist, so kann das Modell ggf. bestimmte Schluss-Kadenzen und -muster erkennen und übernehmen.

3 Preprocessing

Genau gleich wie bei KK

4 Auch hier ein erster Versuch

5 Weitere Versuche

6 Ein etwas anderes Preprocessing

7 Analog zu RNNs

Seq -i [0, 127] fruchtet nicht, max. Acc nach 20 Epochen bei ca. 65prz
Wieso, weshalb, warum?

8 Ein neuer Ansatz

Tokenization der folgenden Form: $(n, v) \mapsto n + (v - 1) \cdot 128$, n: note, v: voice/Stimm Lage (i.e. Bass = 1, Sopran etc.), führt zu 514 vocab size (inkl. EOS, PAD tokens)

Bereits kleine Architektur mit 130k Parametern erreicht ca. 84prz acc, wunderlicherweise sogar nach der ersten -zweiten Epoche. Ab dann plateaued aber die Acc, wieso, weshalb, warum?

9 Größere Architektur

Was verbessert sich im Vergleich zu oben

10 Benchmarks

- Acc - Confidence vs. Seq-Länge